

1. Ecommerce Search Function

Implementation Theory:

The program stores product information such as Product ID, Product Name, and Category in an array of Product objects. The products are first sorted by Product ID to enable binary search. The user enters a Product ID, and the BinarySearch() method repeatedly divides the array into halves until the product is found or the search range becomes empty. If a matching product is found, its details are displayed; otherwise, an appropriate message is shown. The program also validates user input and allows multiple searches until the user types exit.

```
using System;

using System.Collections.Generic;

using System.Linq;

using Ecom;

Console.WriteLine("=== Interactive E-commerce Product Search ===");

Product[] products = {

    new Product(1000, "Laptop", "Electronics"),

    new Product(2000, "Shirt", "Clothing"),

    new Product(3000, "Watch", "Accessories"),

    new Product(4000, "Mobile", "Electronics")

};

products = products.OrderBy(p => p.ProductId).ToArray();

bool keepSearching = true;

while (keepSearching)

{

    Console.Write("\nEnter a Product ID to search for (or type 'exit' to quit): ");
```

```
string userInput = Console.ReadLine();

if (userInput.Trim().Equals("exit", StringComparison.OrdinalIgnoreCase)) {

    keepSearching = false;

    Console.WriteLine("Goodbye!");

    break;

}

if (int.TryParse(userInput, out int searchId))

{

    Console.WriteLine($"Searching for Product ID: {searchId}...");

    Product result = SearchProduct.BinarySearch(products, searchId);

    if (result != null)

    {

        Console.WriteLine($"[FOUND] Name: {result.ProductName} | Category:

{result.Category}");

    }

    else

    {

        Console.WriteLine($"[NOT FOUND] No product matches that ID.");

    }

}

else

{

    Console.WriteLine($"[ERROR] Please enter a valid numerical Product ID.");

}

}
```

```
namespace Ecom
{
    public class Product
    {
        public int ProductId { get; set; }
        public string ProductName { get; set; }
        public string Category { get; set; }
        public Product(int productId, string productName, string category)
        {
            ProductId = productId;
            ProductName = productName;
            Category = category;
        }
    }
}
```

```
namespace Ecom
{
    public static class SearchProduct
    {
        public static Product BinarySearch(Product[] products, int targetId)
        {
            int low = 0;
            int high = products.Length - 1;
            while (low <= high)
            {
```

```

        int mid = low + (high - low) / 2;

        if (products[mid].ProductId == targetId)
        {
            return products[mid];
        }

        else if (products[mid].ProductId < targetId)
        {
            low = mid + 1;
        }

        else
        {
            high = mid - 1;
        }
    }

    return null;
}
}

```

OUTPUT:

```

Enter a Product ID to search for (or type 'exit' to quit): 1001

Searching for Product ID: 1001...
[NOT FOUND] No product matches that ID.

Enter a Product ID to search for (or type 'exit' to quit): 1000

Searching for Product ID: 1000...
[FOUND] Name: Laptop | Category: Electronics

Enter a Product ID to search for (or type 'exit' to quit):

```

2.Financial Forecasting

Implementation Theory:

The program accepts the starting investment amount, annual growth rate, and number of years from the user. The `GenerateGrowthForecast()` method calculates the investment value for each year by repeatedly multiplying the current value by $(1 + \text{growthRate})$ inside a loop. Each calculated value is stored in a `List<double>` and displayed as the financial forecast timeline. Input validation is also performed to ensure that only valid numeric values are processed.

Execution :

```
using System;

using System.Collections.Generic;

using DataStructures.Algorithms;

Console.WriteLine("=== Dynamic Financial Forecasting ===");

Console.Write("Enter starting investment amount (e.g., 1000): ");

if (!double.TryParse(Console.ReadLine(), out double startingPrincipal)) {

    Console.WriteLine("Invalid amount entered. Exiting.");

    return;

}

Console.Write("Enter annual growth rate as a decimal (e.g., 0.07 for 7%): ");

if (!double.TryParse(Console.ReadLine(), out double annualGrowth)) {

    Console.WriteLine("Invalid growth rate entered. Exiting.");

    return;

}
```

```

Console.WriteLine("Enter the number of years to forecast: ");
if (!int.TryParse(Console.ReadLine(), out int years))
{
    Console.WriteLine("Invalid number of years entered. Exiting.");
    return;
}
Console.WriteLine("\nCalculating projection...\n");
List<double> projection = FinancialForecasting.GenerateGrowthForecast(startingPrincipal,
annualGrowth, years);
for (int i = 0; i < projection.Count; i++)
{
    Console.WriteLine($"Year {i}: ${projection[i]}");
}
namespace DataStructures.Algorithms
{
    public class FinancialForecasting
    {
        public static List<double> GenerateGrowthForecast(double
growthRate, int periods)
        {
            if (periods < 0) throw new ArgumentException("Periods cannot be negative.");
            var forecastTimeline = new List<double>();
            double currentValue = presentValue;
            forecastTimeline.Add(currentValue);
            for (int i = 1; i <= periods; i++)
            {
                currentValue *= (1 + growthRate);
            }
        }
    }
}

```

```
        forecastTimeline.Add(Math.Round(currentValue, 2));
    }

    return forecastTimeline;
}

}
```

OUTPUT:

```
=== Dynamic Financial Forecasting ===
Enter starting investment amount (e.g., 1000): 2000
Enter annual growth rate as a decimal (e.g., 0.07 for 7%): 0.05
Enter the number of years to forecast: 4

Calculating projection...

Year 0: $2000
Year 1: $2100
Year 2: $2205
Year 3: $2315.25
Year 4: $2431.01

...Program finished with exit code 0
Press ENTER to exit console.
```